ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/06/26

Maeve Gualtieri-Reed

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

Set up

```
#Load/install required package here
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
library(tseries)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.2     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

```
library(Kendall)
```

Importing and processing the data set

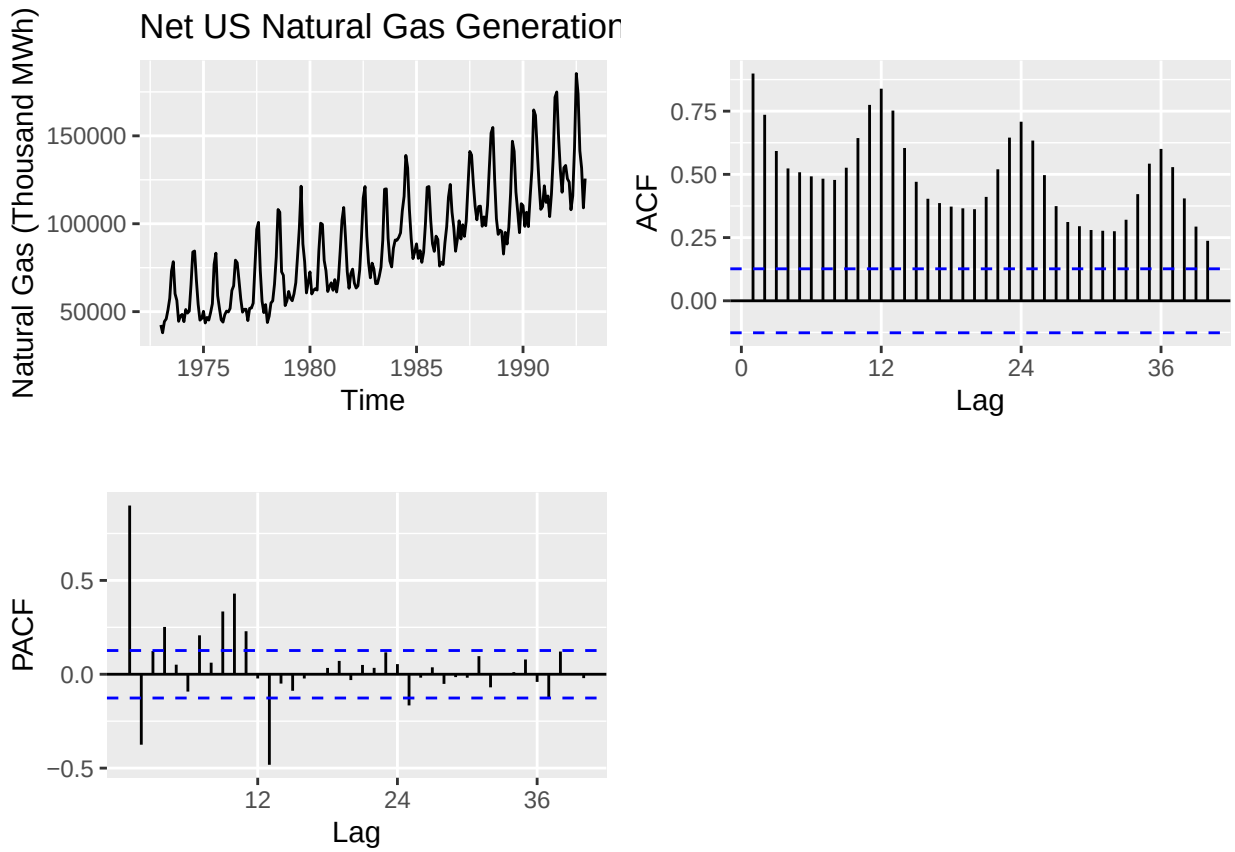
Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

Q1

Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

```
net_generation <- read.csv(file="./Data/Net_generation_United_States_all_sectors_monthly.csv",header=TRU

ng_generation <-
  net_generation %>%
  select(Month, natural.gas.thousand.megawatthours) %>%
  mutate( Month = my(Month)) %>%
  rename( NG = natural.gas.thousand.megawatthours) %>%
  arrange(Month)

nvar <- ncol(ng_generation) - 1
nobs <- nrow(ng_generation)

ts_ng <- ng_generation %>%
  select(NG) %>%
  ts(start=c(1973,1),frequency=12)

plot_grid(
  autoplot(ts_ng) +
    xlab("Time") +
    ylab("Natural Gas (Thousand MWh)")+
    ggtitle("Net US Natural Gas Generation"),
  autoplot(Acf(ts_ng, lag.max = 40, plot=FALSE))+
    ggtitle (""),
```

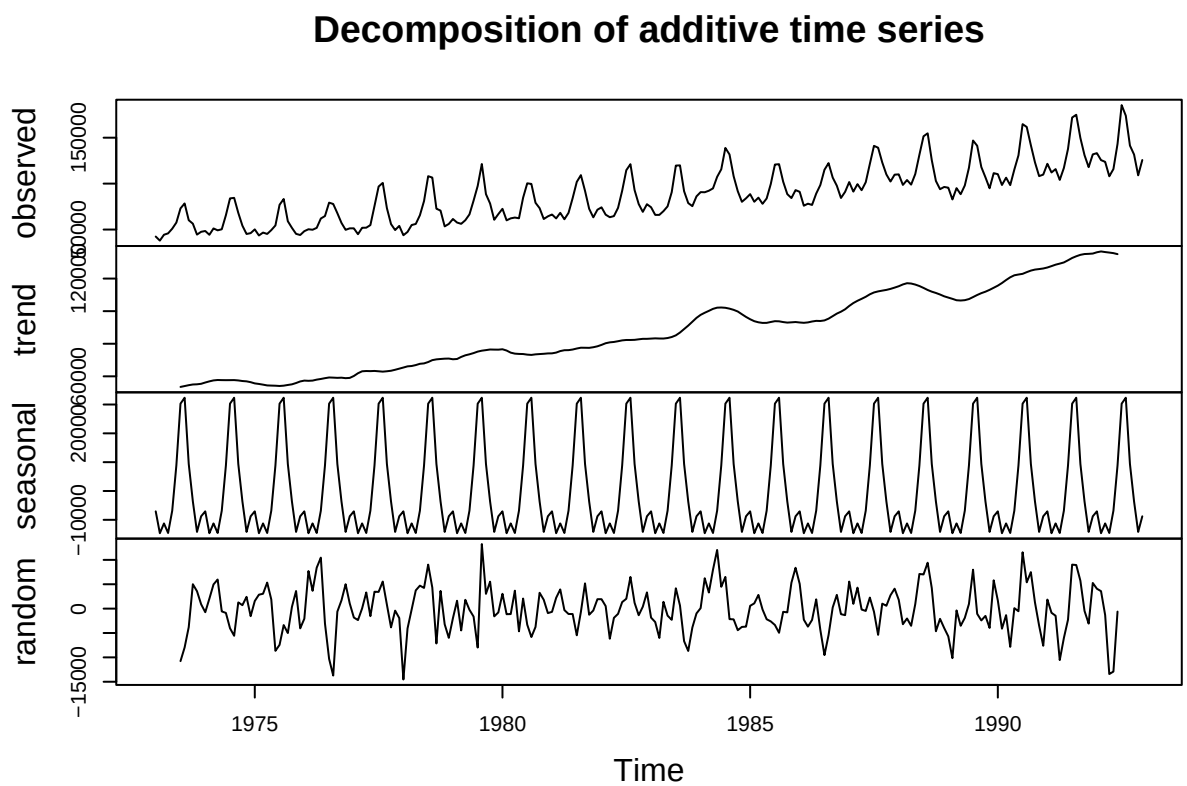
```
  autoplot(Pacf(ts_ng, lag.max = 40, plot=FALSE))+
    ggtitle ("")
)
```



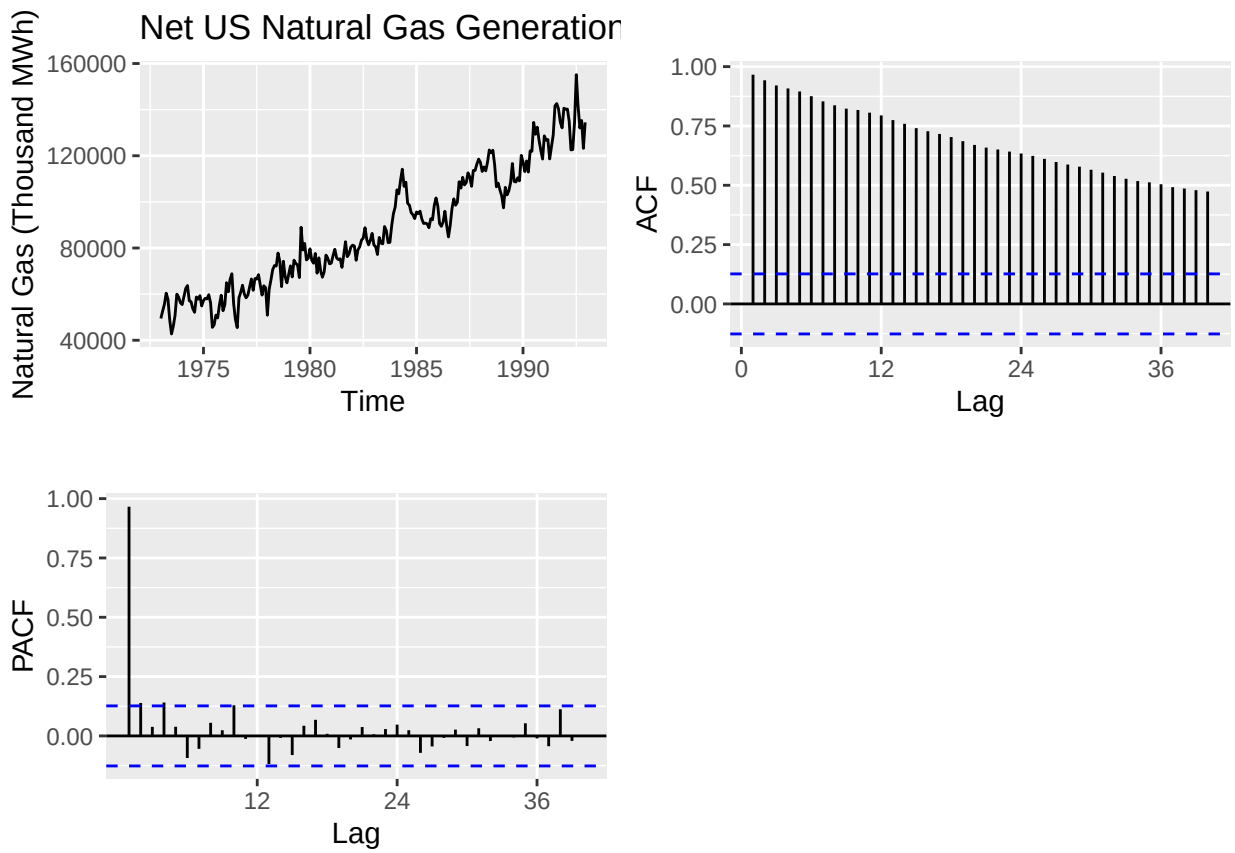
Q2

Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

```
decompose_ng <- decompose(ts_ng,"additive")
plot(decompose_ng)
```




```
deseasonal_ng<- seasadj(decompose_ng)

plot_grid(
  autoplot(deseasonal_ng) +
    xlab("Time") +
    ylab("Natural Gas (Thousand MWh)")+
    ggtitle("Net US Natural Gas Generation - Deseasoned"),
  autoplot(Acf(deseasonal_ng, lag.max = 40, plot=FALSE))+
    ggtitle (""),
  autoplot(Pacf(deseasonal_ng, lag.max = 40, plot=FALSE))+
    ggtitle (")
)
```



The deseasoned plotted series still shows an upward trend and some variability but the clear large peak followed by smaller peak pattern is no longer there. The ACF does not have the seasonal fluctuations seen in the seasoned ACF and the deseasonal PACF has a clear cut off after lag 1 with no peaks outside of the blue dashed lines after. There is still a clear trend component that we will need to difference (the ACF shows a slow decay rather than an exponential)

Modeling the seasonally adjusted (i.e., the deseasonalized series)

Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

```
SMK_deseason<- SeasonalMannKendall(deseasonal_ng)
SMK_deseason
```

```
## tau = 0.887, 2-sided pvalue =< 2.22e-16
```

```
ADF_deseason<- adf.test(deseasonal_ng, alternative = "stationary")
```

```
## Warning in adf.test(deseasonal_ng, alternative = "stationary"): p-value smaller
## than printed p-value
```

```
ADF_deseason
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  deseasonal_ng
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

The seasonal Mann Kendall resulted in a significant p value indicating that there is a trend in the data. However we need to use the ADF to determine if the series is a stochastic trend. The ADF resulted in a significant p value which means we will reject our null hypothesis that there is a unit root and conclude that this is a stationary trend. Because it is a stationary trend, I can either use differencing or regression.

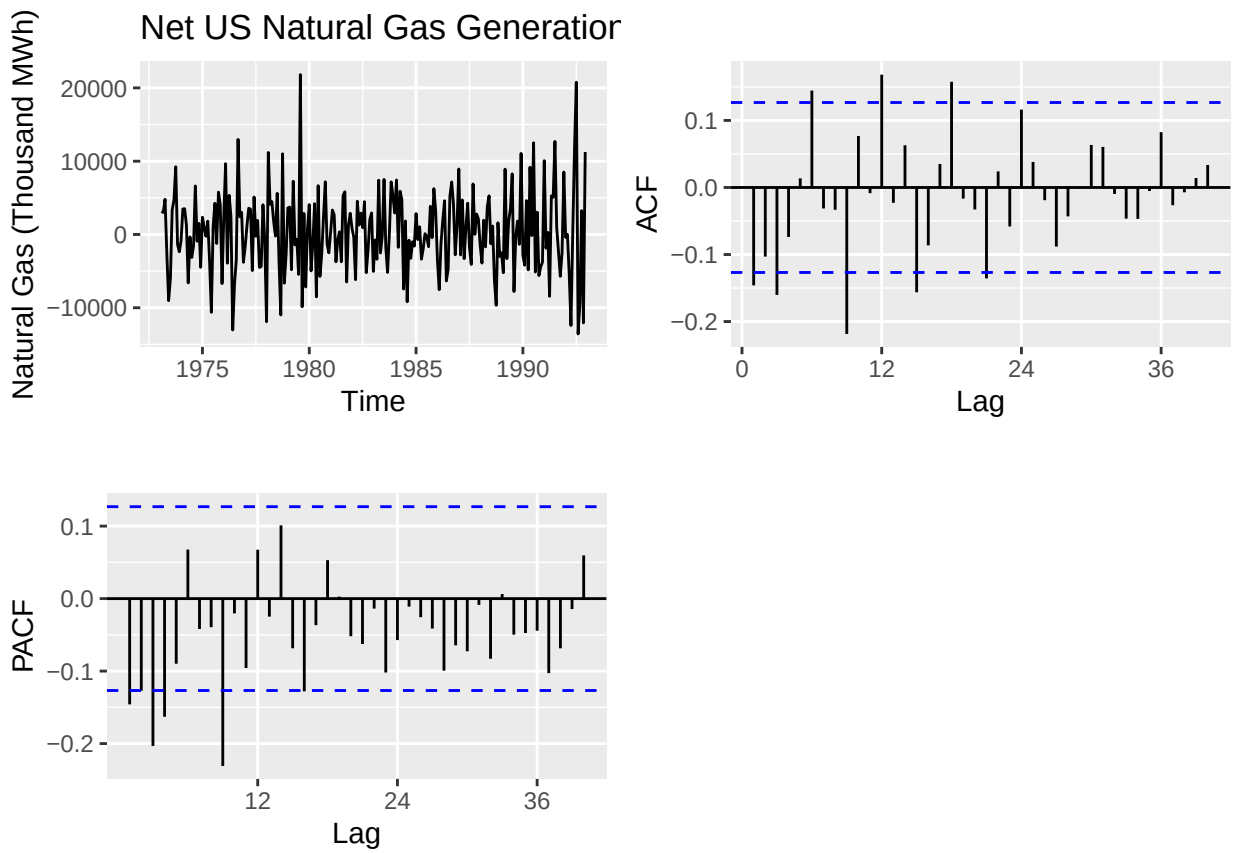
Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

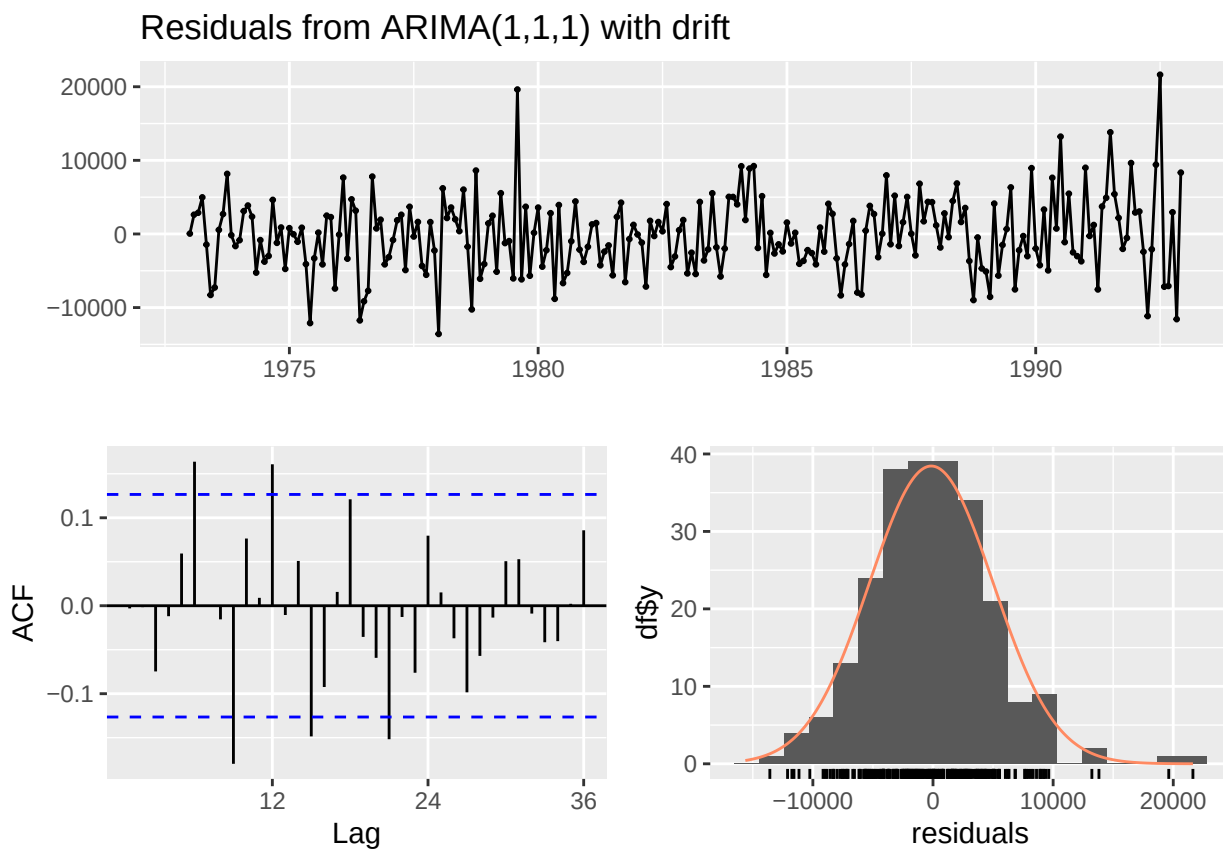
```
# Differencing is needed since the ACF shows a slow decay instead of exponential, but need to determine
n_diff <- ndiffs(deseasonal_ng)
cat("Number of differencing needed: ",n_diff)
```

```
## Number of differencing needed:  1
```

```
deseasonal_diff_ng <- diff(deseasonal_ng,differences=n_diff,lag=1)
plot_grid(
  autoplot(deseasonal_diff_ng) +
    xlab("Time") +
    ylab("Natural Gas (Thousand MWh)")+
    ggtitle("Net US Natural Gas Generation - Deseasoned and Differenced"),
  autoplot(Acf(deseasonal_diff_ng, lag.max = 40, plot=FALSE))+
    ggtitle (""),
  autoplot(Pacf(deseasonal_diff_ng, lag.max = 40, plot=FALSE))+
    ggtitle ("")
)
```




```
#I don't see a cut off in ACF or PACF and exponential trend in the other, so this must have AR and MA c

#ARIMA (1,1,1)
Model_111 <- Arima(deseasonal_ng,order=c(1,1,1),include.drift=TRUE)
compare_aic <- data.frame(Model_111$aic)
checkresiduals(Model_111)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1) with drift
## Q* = 48.356, df = 22, p-value = 0.0009736
##
## Model df: 2.   Total lags used: 24
```

```r
#ARIMA(0,1,1)
Model_011 <- Arima(deseasonal_ng,order=c(0,1,1),include.drift=TRUE)
compare_aic <-  data.frame(compare_aic,Model_011$aic)

#ARIMA(1,1,0)
Model_110 <- Arima(deseasonal_ng,order=c(1,1,0),include.drift=TRUE)
compare_aic <-  data.frame(compare_aic,Model_110$aic)

#ARIMA(2,1,1)
Model_211 <- Arima(deseasonal_ng,order=c(2,1,1),include.drift=TRUE)
compare_aic <-  data.frame(compare_aic,Model_211$aic)

#ARIMA(1,1,2)
Model_112 <- Arima(deseasonal_ng,order=c(1,1,2),include.drift=TRUE)
compare_aic <-  data.frame(compare_aic,Model_112$aic)
```

I know that d = 1 because the ADF and Mann-Kendall tests indicated that there is a stationary trend and ndiff() told me diff = 1, but because the PACF and ACF did not show a clear cut off/exponential decay,

I tested a few models and compared their AIC values to determine p and q. The ARIMA(1,1,1) had the lowest AIC and its residuals seem to be randomly and normally distributed. Therefore p = 1, d = 1, q = 1.
Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

Modeling the original series (with seasonality)

Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., P, D and Q.

Q8

Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.

Checking your model with the auto.arima()

Please do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?